Trustworthy Industrial Anomaly Detection Under Distribution Shift

Technical analysis of the VisA PCB1 implementation

Abstract

This report provides a technical analysis of the provided Python implementation for trustworthy industrial anomaly detection on the VisA PCB1 category. The source is a Colab-oriented, cell-structured Python script focused on leakage-free anomaly scoring, calibration, robustness stress-testing, and multi-scale representation. The implementation reconstructs the PCB1 dataset from verified Parquet files, extracts frozen image representations with pretrained ResNet-50 and DINOv2 ViT-S/14 backbones, constructs a normal reference bank from training data, assigns anomaly scores using cosine-distance k-nearest-neighbor scoring, and maps those scores to probabilities using logistic and isotonic calibration learned without access to the real test defects.

The analysis follows the code as implemented rather than replacing it with an idealized textbook pipeline. It focuses on the separation of reference and calibration data, the use of synthetic calibration corruptions, the mathematical form of the anomaly score and calibration functions, the adaptive expected calibration error implementation, risk-coverage analysis, hyperparameter sensitivity, robustness evaluation under image corruption, and the packaging of generated artifacts. The analysis also identifies implementation constraints that matter for scientific interpretation, including partial rather than strict determinism, environment assumptions, feature-cache invalidation risks, the mismatch between the `ece15` output name and the ten-bin implementation, and the altered class prevalence induced by the synthetic calibration construction.

No experimental results, figures, screenshots, or plots are reproduced in this report. The document is intentionally focused on computational understanding, methodology, mathematical formulation, software structure, and reproducibility.

1. Introduction

Industrial visual anomaly detection addresses a setting in which a model is expected to recognize defective or otherwise unusual observations even though examples of every possible defect are not available during model development. The provided implementation approaches this problem as a one-class reference-learning task: normal training images provide a representation of expected appearance, and test images are assigned anomaly scores according to their feature-space distance from that normal reference set.

The implementation is explicitly framed as a trustworthy anomaly-detection pipeline under distribution shift. That framing is reflected in the code structure. The script does not simply train a classifier on the available labels. Instead, it separates three roles that are often conflated in anomaly-detection experiments: a normal reference bank for defining the nominal appearance manifold, a separate calibration pool for learning a score-to-probability mapping, and a held-out test split containing

real industrial defects for final evaluation. Synthetic defects are introduced only in the calibration pool so that probability calibration can be learned without using the real test anomalies.

The computational pipeline combines standard computer-vision backbones with non-parametric anomaly scoring. ResNet-50 is used as a frozen feature extractor after removal of its classification layer, while DINOv2 ViT-S/14 is used through its feature-extraction interface. For DINOv2, the implementation concatenates the normalized class token with the mean of the normalized patch tokens, producing a representation that combines global and aggregated local information. All resulting embeddings are L2-normalized, making the subsequent inner product equivalent to cosine similarity. The k-nearest-neighbor anomaly score is then defined as the average cosine distance to the nearest elements of the normal reference bank.

Calibration is treated as a separate modeling layer. The raw anomaly score is not itself interpreted as a probability. Instead, the implementation fits both a parametric logistic mapping and a non-parametric isotonic mapping on a calibration set consisting of clean normal images and several synthetic defect variants. This distinction is scientifically useful because discrimination and calibration answer different questions: discrimination asks whether anomalies receive larger scores than normal observations, whereas calibration asks whether numerical probabilities have the intended probabilistic interpretation.

The rest of the report reconstructs the implementation from the source code, beginning with environment preparation and dataset reconstruction and then moving through preprocessing, representation learning, anomaly scoring, calibration, evaluation, robustness analysis, and artifact packaging. The emphasis is on code-to-concept traceability and on distinguishing what the program guarantees from what the surrounding comments merely intend.

2. Project Overview

Conceptually, the implementation consists of the following stages:

1. Prepare a Colab-compatible software environment and reinitialize Pillow.
2. Create a project directory tree for data, results, predictions, logs, models, and checkpoints.
3. Fix a small set of seeds and select execution-device parameters.
4. Download the compact VisA PCB1 Parquet files from Hugging Face and verify their SHA-256 hashes.
5. Reconstruct image and mask files from binary Parquet fields and generate a standardized `1cls.csv` index.
6. Define deterministic image preprocessing and two families of image corruptions.
7. Load frozen pretrained ResNet-50 and DINOv2 ViT-S/14 backbones.
8. Extract and cache clean embeddings for the full reconstructed dataset.
9. For each experimental seed, split normal training data into a reference bank and a separate calibration pool.
10. Compute k-nearest-neighbor anomaly scores against the reference bank.
11. Construct synthetic calibration anomalies from the calibration pool and fit logistic and isotonic score calibrators.
12. Evaluate raw anomaly scores and calibrated probabilities on untouched real VisA test samples.
13. Repeat the core experiment across two backbones and three seeds.
14. Examine the sensitivity of DINOv2 scoring to reference-bank size and the number of neighbors.
15. Stress-test the detector under blur, brightness, JPEG compression, and resolution degradation

at three severity levels.
16. Produce reliability and qualitative error-analysis artifacts.
17. Package tables, predictions, logs, figures, and the standardized split file into a ZIP archive.

The central methodological path can be expressed mathematically as

$$x \xrightarrow{f_\theta} z \xrightarrow{\ell_2\text{-normalize}} \hat{z} \xrightarrow{\text{kNN against } \mathcal{B}} s(x) \xrightarrow{g} p(x),$$

where x is an image, f_θ is a frozen backbone, $\hat{z}$ is its normalized embedding, $\mathcal{B}$ is a reference bank of normal embeddings, $s(x)$ is the raw anomaly score, and g is one of the fitted calibration functions that maps a scalar score to an estimated anomaly probability.

The key architectural point is that the code does not learn a feature extractor and an anomaly decision rule jointly. The backbones are frozen, and the learned part of the pipeline is limited to calibration functions applied to the scalar anomaly score. This makes the experimental question narrower and more interpretable: how useful are the pretrained representations for nearest-neighbor anomaly discrimination, and how well can the resulting distance scale be converted into a probabilistic output under the chosen calibration protocol?

3. Technical Foundations

3.1 One-class anomaly detection in representation space

Let $\mathcal{X}_{\text{normal}}$ denote the distribution of nominal images. A representation model $f_\theta : \mathcal{X} \to \mathbb{R}^d$ maps each image into a feature vector. Instead of fitting a conventional binary classifier, the implementation stores a finite reference set

$$\mathcal{B} = \{\hat{z}_1, \hat{z}_2, \ldots, \hat{z}_B\},$$

where each $\hat{z}_j$ is a normalized embedding of a normal training image.

For a query embedding $\hat{z}$, small distances to members of $\mathcal{B}$ indicate local agreement with known normal appearance. Large distances indicate that the query is unlike nearby reference examples and is therefore treated as more anomalous. This is a local density or neighborhood-consistency view of anomaly detection rather than a parametric estimate of a complete normal-data distribution.

3.2 Frozen transfer representations

The feature extractors are pretrained models imported from external libraries. The source code places both models in evaluation mode and disables gradients for every parameter. Consequently, the experiment is not a fine-tuning study. It is a frozen-representation study.

The ResNet branch replaces the final fully connected classification layer with an identity mapping. Therefore, the network output is the representation immediately before the original ImageNet classifier.

The DINOv2 branch uses `forward_features` and constructs

$$z_{\text{DINO}} = \left[z_{\text{CLS}} \parallel \frac{1}{P} \sum_{p=1}^{P} z_{\text{patch},p} \right],$$

where z_{CLS} is the class-token representation, $z_{\text{patch},p}$ is the representation of patch p, P is the number of patch tokens, and $\parallel$ denotes concatenation. This should be read as a two-source representation: one global token and one mean-pooled summary of local patch representations. Although the source describes this as a multi-scale representation, it is not a conventional image-pyramid or multi-resolution feature hierarchy; the implementation uses two token-derived summaries from a single ViT forward pass.

3.3 Normalization and cosine geometry

The code applies

$$\hat{z} = \frac{z}{\|z\|_2}$$

to every extracted embedding. For normalized vectors, the Euclidean inner product satisfies

$$\hat{z}_i^\top \hat{z}_j = \cos(\angle(\hat{z}_i, \hat{z}_j)).$$

The implementation therefore defines the pairwise distance as

$$d(\hat{z}_i, \hat{z}_j) = 1 - \hat{z}_i^\top \hat{z}_j.$$

This is a cosine-distance formulation on L2-normalized features. It is especially convenient here because the entire query-to-reference similarity matrix can be computed as a matrix multiplication.

3.4 k-nearest-neighbor anomaly scoring

For a query embedding $\hat{z}$, let $d_{(1)}(\hat{z}), \ldots, d_{(k)}(\hat{z})$ be the k smallest distances to the reference bank. The anomaly score is

$$s(\hat{z}) = \frac{1}{k} \sum_{j=1}^{k} d_{(j)}(\hat{z}).$$

Higher values indicate that even the nearest normal reference examples are relatively distant.

The use of an average over the nearest k neighbors makes the score less sensitive than a one-neighbor rule to one unusually similar or dissimilar reference image. The code exposes k as a hyperparameter and uses $k = 5$ for the main experiment.

3.5 Calibration as a separate probabilistic layer

The raw kNN score is a distance-derived quantity, not a probability. The code therefore fits two one-dimensional calibration models.

The logistic model estimates

$$p_{\log}(y = 1 \mid s) = \sigma(as + b) = \frac{1}{1 + \exp[-(as + b)]},$$

where a and b are fitted parameters and σ is the logistic sigmoid.

The isotonic model instead learns a non-decreasing function

$$p_{\text{iso}}(y = 1 \mid s) = g(s),$$

subject to the monotonicity constraint

$$s_1 \leq s_2 \quad \Rightarrow \quad g(s_1) \leq g(s_2).$$

Monotonicity is natural here because the anomaly score is intended to increase with abnormality.

3.6 Calibration metrics

The implementation evaluates calibration with adaptive expected calibration error, the Brier score, and a risk-coverage calculation.

For adaptive ECE, predictions are partitioned using empirical quantiles of the predicted probabilities. For bin b, let n_b be its sample count, c_b the mean predicted probability, and f_b the empirical anomaly frequency. The reported quantity is

$$\text{ECE} = \sum_{b=1}^{M} \frac{n_b}{N} \left| c_b - f_b \right|.$$

The use of quantile-based bins is intended to reduce empty or extremely sparse bins when the probability distribution is highly concentrated. The implementation falls back to equal-width bins if the quantile construction collapses to too few unique boundaries.

The Brier score is

$$\text{BS} = \frac{1}{N} \sum_{i=1}^{N} \left(p_i - y_i \right)^2.$$

Lower values indicate that predicted probabilities are, on average, closer to the observed binary outcomes.

For risk-coverage analysis, the predicted class is

$$\hat{y}_i = \mathbf{1}[p_i \geq 0.5],$$

and confidence is

$$c_i = \max(p_i, 1 - p_i).$$

Samples are sorted from highest to lowest confidence. At each coverage level, the cumulative classification error defines empirical risk. The code approximates the area under this risk-coverage curve using numerical trapezoidal integration.

3.7 Discrimination metrics

The code uses AUROC and average precision (AUPRC) on the raw anomaly scores. Because larger scores correspond to stronger anomaly evidence, a higher score is treated as a positive anomaly ranking.

AUROC measures ranking quality across possible score thresholds. Average precision summarizes the precision-recall relationship and is sensitive to class prevalence. Neither metric directly establishes probabilistic calibration.

4. Input Data and Reconstruction

4.1 Dataset source and integrity checks

The source obtains compact Parquet files from the Hugging Face dataset repository identified in the code as `BrachioLab/visa`. Separate files are used for training and testing. Before downloading a file, the implementation checks whether a local copy exists and whether its SHA-256 digest matches a hard-coded expected value. If the local digest does not match, the file is deleted and redownloaded.

The digest function reads each file in chunks and updates a SHA-256 hash object incrementally:

$$H = \mathrm{SHA256}\,(\text{bytes of the complete file})\,.$$

This is useful for reproducibility because it checks file identity rather than merely file size or presence.

The implementation copies the verified cached download into the experiment's own data directory. The source of the file is therefore explicit, and the local copy is independently checked after copying.

4.2 Binary image decoding

The Parquet records contain image and mask fields whose exact Python representation may be bytes-like objects or dictionaries containing byte payloads. `_bytes_from_value` normalizes these cases into a `bytes` value. Unsupported representations raise a `TypeError`, which is preferable to silently treating unexpected data structures as valid images.

RGB images are decoded with Pillow, explicitly converted to RGB, and saved as JPEG with quality 95. Masks are decoded as grayscale images and stored as PNG files. The masks are stored only for anomalous samples.

The reconstruction step queries the Parquet files through DuckDB:

```
SELECT image, mask, label FROM read_parquet(...).
```

Each returned record is assigned a deterministic file name containing the split name and row index. The binary class rule is explicitly

$$\text{is\_anomaly} = [\text{label\_id} = 1].$$

Thus, the implementation assumes the source labels use `1` for anomaly and treats the alternative label value as normal.

4.3 Standardized split index

The reconstruction process produces a row-level table with object name, split, string label, image path, mask path, and integer label. The table is written to `1cls.csv`, providing a stable index for subsequent processing.

The code subsequently filters this table rather than re-reading the raw Parquet files. In the main experiment, only images with `split == 'train'` and `label == 'normal'` are eligible for the reference and calibration pools. The test frame includes all test images, including real anomalies.

This separation is central to the leakage-free design. The real test labels are used only for evaluation, after the reference bank and calibrators have been fixed.

5. Preprocessing and Synthetic Corruptions

5.1 Image preprocessing

The common preprocessing pipeline performs three operations:

$$x \to \text{Resize}(x, 224, 224) \to \text{ToTensor}(x) \to \text{Normalize}(x; \mu, \sigma),$$

with ImageNet channel statistics

$$\mu = (0.485, 0.456, 0.406), \qquad \sigma = (0.229, 0.224, 0.225).$$

The fixed square resize means that original aspect ratio is not preserved. The code does not apply a center crop, random crop, or explicit geometric normalization. Consequently, all inputs are forced to the same spatial resolution before entering either pretrained backbone.

5.2 Synthetic defects for calibration

The calibration stage is unusual in an important way: it needs positive anomaly examples even though the true anomalies in the held-out test set cannot be used for fitting. The implementation addresses this by perturbing normal calibration images with controlled synthetic defects.

Four calibration defect families are implemented.

Subtle additive noise

Pixel values are converted to the interval $[0, 1]$. A random Gaussian perturbation with standard deviation sampled uniformly from 0.015 to 0.035 is added and clipped to $[0, 1]$:

$$x' = \text{clip}(x + \epsilon, 0, 1), \qquad \epsilon \sim \mathcal{N}(0, \sigma^2 I).$$

The code samples a new σ for each image.

Cut-and-paste patch

A patch with width and height independently sampled from the integer range 12 to 27 pixels is taken from one image location and copied to another. With probability one half, the patch is rotated by 180° before insertion. This creates a localized appearance inconsistency while preserving most of the image.

Subtle blur

A Gaussian blur radius is sampled uniformly from 0.4 to 0.9. This introduces a localized-defect calibration example only in the broad sense that the transformed image is presented as a positive synthetic anomaly; spatial localization is not explicitly controlled by this corruption.

Micro-scratch

A short line is generated by selecting a starting point, a random angle, and a length between 15 and 39 pixels. The affected pixels are brightened by a random amount between 60 and 119 intensity units before clipping.

The docstring for the synthetic-defect function states that the resulting calibration scores are guaranteed to occupy a particular numerical range and match the scale of real defects. The implementation itself does not enforce such a score range. The scores emerge from the backbone and reference bank. Therefore, this statement should be interpreted as an intended design claim, not as a mathematical guarantee.

5.3 Stress-test corruptions

A separate corruption function is used for out-of-distribution stress testing. It supports:

- Gaussian blur with radius $0.7s$, where $s \in \{1, 2, 3\}$ is severity.
- Brightness scaling using factors 1.15, 0.75, and 0.50.
- JPEG recompression with qualities 75, 45, and 20.
- Resolution degradation by downscaling to 65%, 40%, or 25% of the original dimensions and resizing back to the original size.

These transformations are deterministic with respect to the image-level seed and severity parameters used by the dataset wrapper.

The conceptual distinction between the two corruption families matters. Calibration corruptions are training-pool transformations used to fit the probability mapping, while stress corruptions are applied to the untouched test set after the calibrators have been fit. The latter therefore test behavior under distribution shift rather than augmenting the fitting process.

6. Dataset Abstraction and Execution Model

`PathImageDataset` implements the PyTorch `Dataset` interface. It stores a copy of the incoming dataframe, the common transform, optional corruption settings, and a base seed.

For an index i, `__getitem__` performs:

$$\begin{aligned}\text{row}_i \rightarrow \text{read image} \rightarrow \text{RGB conversion} \rightarrow \text{optional corruption} \\ \rightarrow \text{tensor transform} \rightarrow (\text{image tensor}, \text{label}, \text{path}).\end{aligned}$$

The dataset passes a seed of the form

$$s_i = s_{\text{base}} + i$$

to corruption routines. This makes the image-level transformation deterministic for a fixed dataset ordering and seed.

The `DataLoader` uses `shuffle=False`, which is important because the returned paths are later used to map embeddings back to dataframe rows. The loader also uses pinned memory when the GPU is active and chooses the worker count based on the execution device.

One subtle design decision is that feature extraction is path-indexed after the fact. The code extracts embeddings in loader order, saves the paths alongside those embeddings, and later constructs a dictionary from path to feature-row index. This avoids assuming that dataframe row order and feature-array order are permanently interchangeable.

7. Backbone Loading and Feature Extraction

7.1 ResNet-50 branch

The ResNet-50 model is loaded with the library's default pretrained weights. Its final classification layer is replaced with `nn.Identity()`:

$$f_\theta^{\text{ResNet}}(x) = h_\theta(x),$$

where $h_\theta(x)$ is the feature vector immediately preceding the original classifier.

The model is placed on the selected device and switched to evaluation mode. Every parameter is marked `requires_grad=False`, so no optimization graph is constructed for the model.

This is a conventional way to repurpose a classification network as a fixed representation function.

7.2 DINOv2 branch

DINOv2 is loaded through `torch.hub` using the `dinov2_vits14` entry point. The same frozen-model treatment is applied.

The source explicitly accesses the dictionary returned by `forward_features` and reads the normalized class token and normalized patch tokens. Patch tokens are averaged along the patch dimension:

$$\bar{z}_{\text{patch}} = \frac{1}{P}\sum_{p=1}^{P} z_{\text{patch},p},$$

then concatenated with the class token.

The code therefore creates a representation that combines a global semantic token with the average of local patch-level representations. The patch mean is computationally simple, but it deliberately discards the spatial arrangement among patches after aggregation.

7.3 Mixed-precision inference

When CUDA is active, the source wraps the forward pass in `torch.amp.autocast('cuda')`. CPU execution uses a null context.

After inference, embeddings are converted to float32 and normalized before they are copied to CPU NumPy arrays. This arrangement reduces intermediate GPU memory pressure while ensuring a consistent float32 representation for the downstream NumPy calculations.

7.4 Feature caching

Clean features are cached as a compressed `.npz` file together with a CSV containing the corresponding paths and labels.

Caching is beneficial because the same frozen representations are reused across multiple seeds, calibration trials, hyperparameter checks, and robustness experiments. The cache is particularly valuable when the backbones are the computational bottleneck.

However, the cache key is effectively just the backbone name. The cache does not record the source-image hash, preprocessing configuration, model weight identity, library version, or feature dimensionality. A stale cache could therefore be reused after a change in model or preprocessing configuration if the expected files happen to remain present. This is an implementation-level reproducibility limitation.

8. Leakage-Free Reference and Calibration Protocol

8.1 Category filtering

`get_category_frames` constructs two frames for the selected object category. The training frame contains only normal examples; the test frame contains all test examples.

In this configuration, the category list contains only `pcb1`. Thus, although the helper functions accept a category argument, the actual experiment is single-category rather than a multi-category benchmark.

8.2 Reference and calibration split

The normal training frame is randomly permuted using a NumPy generator seeded with the trial seed. A maximum of 250 samples is assigned to the reference bank and a further maximum of 150 samples is assigned to the calibration pool.

The split is without replacement because the calibration indices are taken from the unused portion after the bank indices have been selected.

Formally, for training normals $\mathcal{T}$,

$$\mathcal{T} = \mathcal{B}_{\text{data}} \cup \mathcal{C}_{\text{data}} \cup \mathcal{R},$$

where $\mathcal{B}_{\text{data}}$ is the reference subset, $\mathcal{C}_{\text{data}}$ is the calibration subset, and $\mathcal{R}$ is the remainder not used by the main method.

The separation is scientifically important. If the same examples defined both the normal reference bank and the calibration mapping, the calibration score distribution would be tied to the exact points used to define normality. The code avoids that coupling by construction.

8.3 Converting dataframe rows to feature rows

The feature cache contains an array and a parallel list of paths. `frame_to_feature_rows` constructs a path-to-index dictionary and retrieves the correct feature rows for a selected dataframe.

This path-based indexing is an explicit data-integrity safeguard. It reduces the chance that later dataframe filtering or concatenation silently changes the association between metadata and embeddings.

9. Anomaly Scoring and Calibration

9.1 Chunked similarity computation

`score_knn` processes query embeddings in chunks. For a query block Q and reference matrix R, it computes

$$S = QR^{\top},$$

where each matrix element is the inner product between one normalized query and one normalized reference vector. Because the vectors are normalized,

$$D = \mathbf{1} - S$$

is the corresponding cosine-distance matrix.

For every query row, the implementation uses `np.partition` to obtain the k smallest distances without fully sorting all reference distances. It then averages those nearest values.

This is efficient for the intended use case. A full sort would perform more work than necessary when only the first k elements are required.

9.2 Calibration data construction

For a trial, the calibration score set consists of:

1. scores from clean normal calibration images, labeled 0;
2. scores from cut-and-paste calibration variants, labeled 1;
3. scores from subtle-noise variants, labeled 1;
4. scores from subtle-blur variants, labeled 1;
5. scores from micro-scratch variants, labeled 1.

With the configured calibration-pool size of 150, the intended construction contains up to 150 clean examples and up to 4×150 synthetic positive examples, provided the source pool is large enough.

The key methodological property is not the exact sample count but the information boundary: real test images never contribute to the calibration fit.

9.3 Logistic calibration

The logistic regression uses a single predictor: the scalar anomaly score. Its fitted decision function can be written as

$$\eta(s) = as + b,$$

with probability

$$p_{\log}(s) = \sigma(\eta(s)).$$

The regularization parameter is explicitly set to $C = 1.0$, the solver is L-BFGS, and the maximum iteration count is 1000.

The model is therefore intentionally low-dimensional. It does not learn a representation; it learns only a monotone sigmoid-like remapping of score magnitude.

9.4 Isotonic calibration

The isotonic regressor fits a monotone function directly to the calibration pairs (s_i, y_i). Its output is constrained to the interval $[0, 1]$ and is clipped for out-of-bounds scores.

Isotonic calibration is more flexible than logistic calibration because it does not impose a particular sigmoid shape. At the same time, its flexibility can make it sensitive to calibration-set composition, especially when the calibration sample is not large.

10. Evaluation Procedure

10.1 Main trial

The core experiment is repeated over

$$3 \text{ seeds} \times 2 \text{ backbones} \times 1 \text{ category},$$

giving six main trials.

For each trial, the reference bank and calibration pool are resampled from normal training data according to the trial seed. The calibrators are fitted only on that trial's calibration data.

The real test images are then scored against the reference bank. AUROC and AUPRC are computed from the raw anomaly scores, while ECE, Brier score, and AURC are computed separately for logistic and isotonic probabilities.

The raw score is therefore the object of discrimination evaluation, and the calibrated probability is the object of probabilistic evaluation.

10.2 What is and is not aggregated

The source stores one result row per backbone-category-seed combination and computes means and standard deviations grouped by backbone for selected metrics. These summaries are aggregation operations, not uncertainty estimates in the inferential-statistical sense. The code does not compute confidence intervals, hypothesis tests, bootstrap intervals, or repeated nested resampling.

The resulting standard deviation across three seeds should therefore be interpreted as a small-sample estimate of variability under the selected random-split protocol, not as a formal population-level uncertainty bound.

10.3 Hyperparameter sensitivity

The reference-bank size B is varied over

$$B \in \{50, 150, 250\},$$

and the neighbor count k is varied over

$$k \in \{1, 5, 10\}.$$

This creates nine configurations evaluated under three seeds, using DINOv2 features.

The sensitivity study is informative about how the kNN detector responds to memory budget and neighborhood size. However, the same real test set is used to compare all configurations. Therefore, this analysis should be viewed as an evaluation or sensitivity study unless an external decision rule has already specified the final hyperparameters. It should not be described as a fully nested hyperparameter-selection procedure.

11. Robustness Stress-Testing Under Distribution Shift

The robustness stage fixes the reference/calibration split to seed 13 and fits calibrators in the same leakage-free manner as the main experiment. It then evaluates the untouched test set under both clean and corrupted conditions.

For each backbone, the stress suite covers four corruption types and three severities, in addition to a clean condition.

The raw anomaly detector is kept unchanged. Only the test images are transformed. Consequently, the experiment probes whether the representation and kNN score maintain useful discrimination when image appearance changes while the underlying normal/anomalous label remains fixed.

The calibrated probability is also passed through the same logistic or isotonic mapping learned from clean plus subtle synthetic calibration data. This is important: the stress experiment does not re-fit the calibrators under the corruption. Calibration degradation is therefore a measure of how the previously learned probability mapping transfers to shifted score distributions.

This protocol separates two failure modes:

$$\text{distribution shift} \rightarrow \begin{cases} \text{ranking degradation,} \\ \text{probability-calibration degradation.} \end{cases}$$

A model may preserve AUROC while becoming poorly calibrated, or it may lose both ranking and calibration quality. The source explicitly records both families of measures.

12. Reliability Analysis and Qualitative Error Diagnosis

12.1 Reliability diagram

The reliability-diagram function evaluates the DINOv2 seed-13 predictions using adaptive calibration bins. It compares mean predicted probability with empirical anomaly frequency.

Although the source generates a graphical reliability diagram and histogram, this report deliberately does not reproduce those figures. The underlying computational logic is still important because it provides a visual companion to ECE: ECE is a single scalar, whereas a reliability relationship reveals where along the probability axis the model is over- or under-confident.

12.2 Ablation table

The ablation table records three conceptual variants per backbone:

$$\text{raw score}, \qquad \text{score + logistic calibration}, \qquad \text{score + isotonic calibration}.$$

The AUROC and AUPRC values are copied unchanged across the three variants because calibration is applied after the raw ranking has been produced. ECE and Brier score are only reported for the calibrated variants.

This separates ranking effects from calibration effects. A monotone calibration transformation can change the numerical interpretation of a score without changing its ordering, so ranking metrics should remain unchanged unless numerical ties or implementation details intervene.

12.3 False-positive and false-negative selection

The qualitative error-analysis code focuses on the DINOv2 seed-13 trial. It selects the four highest-scoring normal test examples as false positives and the four lowest-scoring real anomalies as false negatives.

The selection rule is direct:

$$\text{FP} = \text{Top4}\left\{s_i : y_i = 0\right\},$$

$$\text{FN} = \text{Bottom4}\left\{s_i : y_i = 1\right\}.$$

The source then saves image grids with raw scores and isotonic probabilities. Those images can be useful for expert diagnosis of representation failure modes, but no automated semantic interpretation of the failure cases is performed in the code. Therefore, the report does not assign visual causes to individual errors.

13. Software Architecture and Code Organization

The program is organized as a linear research script with helper functions rather than as a multi-module Python package. This is appropriate for interactive Colab execution but has consequences for maintainability.

The code can be understood as nine functional layers:

1. **Environment reset:** module purging, Pillow replacement, runtime restart.
2. **Configuration:** directory creation, seeds, device, backbone names, batch size, image size.
3. **Dataset reconstruction:** download, hash verification, image/mask materialization, CSV generation.
4. **Synthetic data generation:** calibration and stress corruptions.
5. **Dataset abstraction:** `PathImageDataset`.
6. **Representation extraction:** backbone loading, inference, feature normalization, caching.
7. **Scoring and calibration:** reference sampling, kNN score, calibration models, metrics.
8. **Experiment orchestration:** main trials, sensitivity analysis, robustness analysis, qualitative diagnostics.
9. **Packaging:** result-directory assembly and ZIP creation.

The principal state is kept in module-level variables such as `split_df`, `clean_features`, `MODELS`, and the result dataframes. This makes the script easy to execute top-to-bottom in a notebook-like environment, but it also means individual functions depend on global state rather than explicit dependency injection.

14. Detailed Control Flow

The intended top-to-bottom execution is important because the file is designed for Colab.

The environment cell first detects whether Colab is present. In Colab, the process exits intentionally after reinstalling Pillow. The notebook must then be rerun from the beginning so that the new C-extension is loaded into a fresh process.

Once restarted, the script defines the project directories and imports all computational libraries. It then reconstructs the dataset and writes the split table.

The backbones are loaded only after the dataset abstraction is defined. Clean embeddings are extracted once per backbone and cached. The main experimental loop then reuses those embeddings rather than repeatedly running the frozen networks on clean images.

Synthetic calibration features are extracted inside each trial because the transformed images differ by corruption type and seed. Stress-test features are also extracted on demand because they depend on corruption and severity.

Finally, the script writes tables and predictions, generates diagnostic graphics, and packages the artifacts. The `final_package` directory is rebuilt on every run by deleting any previous directory and recreating it.

This execution order matches the computational dependencies: model loading precedes feature extraction, and feature caching precedes repeated scoring analyses.

15. Design Decisions and Rationale

The rationale below follows from the implementation choices, although the author's intent is not separately documented.

15.1 Frozen backbones rather than task-specific training

Freezing ResNet-50 and DINOv2 isolates the effect of representation quality from the optimization behavior of a trainable anomaly model. It also reduces the risk of overfitting a small industrial dataset to a particular defect taxonomy.

The trade-off is that no feature adaptation is possible. If the pretrained representation does not encode the appearance differences that matter for PCB defects, the downstream kNN score cannot recover that information.

15.2 Reference-bank scoring rather than a parametric density model

The kNN score requires comparatively few assumptions about the shape of normal embeddings. It does not assume Gaussian features, a specific covariance structure, or an explicit probability density.

Its main computational cost grows with the reference-bank size, which motivates the explicit budget experiment.

15.3 Separate calibration pool

The separate calibration pool is one of the strongest experimental design features in the source. It avoids fitting the score-to-probability function on the same normal images used to define the reference neighborhood.

15.4 Synthetic anomalies only for calibration

This permits calibration without consuming real test anomalies. The method is pragmatic, but synthetic calibration examples need not have the same score distribution as real industrial defects. The calibration result is therefore dependent on the realism and coverage of the chosen corruption family.

15.5 Two calibrators

Logistic regression supplies a simple parametric baseline. Isotonic regression supplies a flexible monotone alternative. Together they allow the code to distinguish the question "does a simple sigmoid remapping suffice?" from the question "does a non-parametric monotone mapping better fit the observed calibration relationship?"

15.6 Stress-testing after calibration

Applying distribution-shift corruptions after the calibrators are fixed is methodologically appropriate for evaluating transfer of calibration. Recalibrating on shifted data would answer a different question.

16. Mathematical and Computational Complexity

16.1 Feature extraction

Let N be the number of images, and let C_f denote the computational cost of one backbone forward pass. Feature extraction scales approximately as

$$O(N\,C_f),$$

with the exact constant dominated by the selected backbone and image size.

The use of batching reduces Python-level overhead and can improve GPU utilization. Mixed precision on CUDA can further reduce memory traffic and improve throughput, depending on hardware and PyTorch execution.

16.2 kNN scoring

Let Q be the number of query embeddings, B the reference-bank size, and d the embedding dimension. Computing the similarity matrix costs approximately

$$O(QBd).$$

Finding the smallest k distances with `np.partition` avoids an $O(B \log B)$ full sort for every query row. The exact complexity of the selection primitive depends on the NumPy implementation, but the key design point is that only the nearest k values are required.

Peak temporary memory is controlled by the chunk size C, giving a similarity matrix of roughly $C \times B$ rather than $Q \times B$. The corresponding temporary memory therefore scales as

$$O(CB).$$

16.3 Cache and storage trade-offs

The feature cache trades disk space for repeated computation. This is appropriate because the clean embeddings are reused many times, while the backbones are the most expensive part of the pipeline.

The reconstruction step also expands compact Parquet records into individual JPEG and PNG files. That simplifies downstream image loading and standardizes file-based access, but it increases the number of filesystem objects and introduces a JPEG recompression step.

17. Reproducibility and Reliability

17.1 Positive reproducibility mechanisms

The implementation includes several useful reproducibility practices:

- explicit seed values are defined centrally;
- NumPy random generators are used for dataset splits and corruptions;
- the exact image-transform parameters are in source;
- dataset files are checked with SHA-256 digests;

- clean features are cached with explicit path metadata;
- experiment tables and predictions are written to deterministic locations;
- a standardized split CSV is created from the reconstructed dataset.

These practices improve repeatability and auditability.

17.2 Why reproducibility is not fully deterministic

The source calls

```
torch.backends.cudnn.benchmark = True
```

when initializing seeds. cuDNN benchmarking can select algorithms based on runtime characteristics and is not the same as requesting deterministic kernels. The source does not set `torch.backends.cudnn.deterministic = True` and does not globally disable nondeterministic operations.

Thus, the correct characterization is **seeded and partially controlled**, not strictly bitwise deterministic.

The script also uses multiprocessing in the `DataLoader` when CUDA is available. Although the corruption itself is deterministically parameterized by an image index and base seed, exact end-to-end reproducibility can still depend on worker behavior and library implementations.

17.3 External dependency reproducibility

Pillow is explicitly pinned to `12.3.0`. In contrast, the source does not pin the versions of PyTorch, torchvision, DuckDB, scikit-learn, NumPy, pandas, or the Hugging Face tooling. Their versions are printed at runtime, but printing a version does not by itself recreate an environment.

The DINOv2 weights are loaded through `torch.hub`, and the code does not record a repository commit or model-weight checksum. ResNet uses the library's `DEFAULT` weight specification rather than an explicit immutable artifact identifier.

These choices are common in exploratory Colab workflows, but they limit long-term reproducibility.

17.4 Environment reset as a reliability measure

The Pillow reset logic is unusually aggressive: it purges loaded PIL modules, uninstalls Pillow, manually removes matching site-package directories, reinstalls a chosen version, and terminates the Colab process.

The intent is to ensure that compiled Pillow extensions come from a clean installation. The approach can resolve stale binary-extension problems, but it is also destructive and tightly coupled to a mutable Colab environment. It is not a portable environment-management strategy for ordinary local execution.

18. Limitations and Technical Considerations

18.1 Calibration prevalence is synthetic

The calibration labels are constructed from one clean set and four synthetic anomaly sets. With a calibration pool of 150 images, this can create a nominal calibration class ratio of approximately 1

clean example to 4 synthetic anomaly examples.

That class prevalence is a property of the calibration protocol, not necessarily the real prevalence of defects in deployment. Because the logistic intercept depends on class balance, the calibrated probabilities can inherit a prior-distribution mismatch. Isotonic calibration can also be affected by the calibration score distribution and label frequencies.

Therefore, the resulting probabilities should be interpreted as probabilities under the implemented calibration sample construction rather than as a universally valid estimate of deployment prevalence.

18.2 Synthetic anomalies may not match real defects

Cut-and-paste, noise, blur, and scratch transformations are useful stressors, but they are only proxies for the unknown distribution of real industrial defects. Calibration quality on these synthetic positives therefore depends on whether the synthetic examples occupy a score regime representative of real anomalies.

The code never demonstrates that equivalence. The report consequently treats synthetic calibration as a methodological device rather than as evidence that the synthetic and real defect distributions coincide.

18.3 `ece15` naming mismatch

The result columns are named `logistic_ece15` and `isotonic_ece15`. However, the metric function is called with its default `n_bins=10`. The implementation therefore computes a ten-bin adaptive ECE while naming the output as though it were an ECE-15 metric.

This is a concrete reporting inconsistency. It does not necessarily make the numerical ECE implementation wrong, but it can create confusion when comparing the saved table against documentation or another experiment.

18.4 Main experiments and secondary analyses use different seed coverage

The primary experiment runs three seeds. The robustness stress test, reliability diagram, and qualitative error diagnosis use seed 13 only.

These are therefore single-seed diagnostics rather than three-seed robustness summaries. Their conclusions should not be generalized to the full seed distribution without additional runs.

18.5 Hyperparameter study is not nested

The B-versus-k sensitivity study evaluates multiple configurations against the same real test set. If a configuration were selected on the basis of those test metrics and then presented as the final model, the test set would have served as a tuning resource.

The clean interpretation is a post hoc sensitivity analysis that illustrates how the detector behaves under alternative parameterizations.

18.6 Feature cache invalidation is implicit

The feature-cache files are named by backbone only. Changes in preprocessing, source data, model weight version, or embedding construction could therefore leave a cache file that is technically readable but scientifically stale.

A stronger implementation would store a manifest containing data hashes, preprocessing parameters, model identifiers, feature dimensionality, and perhaps a source-code or configuration hash.

18.7 Image reconstruction introduces JPEG transformation

The raw RGB images are saved as JPEG with quality 95. If the original Parquet data are not already encoded in exactly the same way, this step can change pixel values. The experiment therefore operates on reconstructed JPEG files rather than directly on the original byte representation.

For a representation-based detector this may be acceptable, but it is still a transformation that should be included in a strict reproducibility description.

18.8 Masks are reconstructed but not used in the detector

Anomaly masks are extracted and saved for anomalous images, but the actual scoring, calibration, robustness evaluation, and metrics are image-level. No pixel-level localization metric, mask overlap, or segmentation loss is computed.

The presence of masks in the reconstructed dataset should therefore not be interpreted as evidence that the method performs defect localization.

18.9 Multi-scale terminology should be interpreted carefully

The DINOv2 representation combines a CLS token with a mean of patch tokens. This is richer than using a single token alone, but it is not a conventional multi-scale feature pyramid. The code does not extract feature maps at multiple spatial resolutions.

Calling the method "multi-scale" is therefore reasonable only in the broad sense of combining global and aggregated local token information.

18.10 Packaging message is stronger than the implemented verification

The packaging code writes a `README.txt` stating that all verification checks passed. The shown code verifies that the files can be copied and zipped and reports successful archive creation. It does not implement a comprehensive semantic verification of every table, prediction file, or figure.

The message should therefore be understood as a packaging-success message rather than as evidence of a formal end-to-end artifact audit.

19. Scientific Interpretation Boundaries

A rigorous reading of the implementation requires separating several claims that are easy to conflate.

First, the code **does implement** a leakage-aware protocol in which the real test anomalies are withheld from the calibration fit. That is directly supported by the sequence of data selection, calibration fitting, and final test evaluation.

Second, the code **does implement** score calibration. It does not prove that the resulting probabilities are calibrated under any deployment distribution beyond the empirical conditions represented by the calibration data.

Third, the code **does implement** distribution-shift stress testing. It does not establish invariance to arbitrary industrial shifts, because the shift family is limited to four hand-designed image corruptions at three severities.

Fourth, the source **does compute** several recognized evaluation metrics. The metrics alone do not establish statistical significance, generalization to other categories, or superiority over alternative methods because the shown implementation does not provide a complete comparative benchmark design.

Finally, the source **contains diagnostic plots and result tables**, but this report does not reproduce or interpret their empirical values. No numerical performance claim is made here beyond what can be established from code structure and configuration.

20. Overall Technical Assessment

The implementation is technically coherent as a research prototype for image-level anomaly detection using frozen representations. Its strongest methodological feature is the explicit separation of reference construction, calibration, and final evaluation. The code also demonstrates a useful awareness of practical reproducibility issues through file hashing, controlled split sampling, deterministic corruption parameters, feature caching, and explicit artifact packaging.

The choice of cosine-normalized features plus kNN scoring is computationally straightforward and conceptually transparent. It makes the anomaly score easy to inspect and separates representation extraction from anomaly scoring. This separation is especially valuable for a research-oriented study because changes to the backbone, reference size, or calibration layer can be analyzed independently.

The calibration design is also methodologically disciplined. Logistic and isotonic regression are both applied to the same scalar score, making their comparison relatively interpretable. The stress-test stage goes beyond clean-test discrimination by asking whether both ranking and probability interpretation survive controlled distribution shifts.

At the same time, the implementation is clearly optimized for an interactive Colab research workflow rather than for a fully packaged production or benchmark framework. Environment mutation, partially pinned dependencies, implicit feature-cache validity, and single-seed secondary analyses limit strict reproducibility. The synthetic calibration distribution is another major methodological assumption, particularly because probability calibration can depend on class prevalence and score coverage.

There are also several nomenclature and documentation issues that should be corrected before presenting the code as a finalized scientific benchmark. The `ece15` naming mismatch should be resolved; the "multi-scale" terminology should be stated more precisely; the packaging verification message should not imply checks that are not performed; and the reproducibility configuration would benefit from explicit model and dependency identifiers.

Despite these limitations, the code demonstrates solid research engineering practices. It does not simply train a model and report accuracy. Instead, it constructs a reusable feature pipeline, introduces an explicit leakage boundary, treats calibration as a first-class problem, evaluates distribution shift, measures confidence quality with multiple metrics, and preserves intermediate artifacts for later analysis.

21. Reproducibility Checklist Derived from the Source

A faithful reconstruction of the experiment requires the following conditions to remain fixed:

- Python and all runtime libraries, especially PyTorch, torchvision, NumPy, pandas, scikit-learn, DuckDB, Pillow, and Hugging Face tooling.
- The exact VisA Parquet files identified by their SHA-256 hashes.
- The preprocessing sequence and ImageNet normalization constants.
- The selected backbones and their exact pretrained weight artifacts.
- The reference-bank size $B = 250$, calibration-pool size 150, and main neighbor count $k = 5$.
- The three primary seeds $13, 42, 123$.
- The category configuration containing only `pcb1`.
- The DINOv2 representation construction from CLS plus mean patch tokens.
- L2 normalization before cosine-distance scoring.
- The four synthetic calibration corruptions and their randomization rules.
- Logistic regression configuration and isotonic regression settings.
- Adaptive ECE with the implementation's actual default of ten bins.
- The distinction between clean main evaluation, single-seed robustness evaluation, and single-seed qualitative diagnostics.
- The feature-cache contents and the path-to-row mapping.

For a stronger future reproducibility package, the source should additionally record explicit environment versions, immutable model identifiers, data manifests, configuration hashes, and cache provenance.

Conclusion

The provided implementation constitutes a research-oriented anomaly-detection pipeline built around frozen visual representations, a normal reference bank, nearest-neighbor anomaly scoring, and post-hoc probability calibration. The central computational idea is simple but useful: convert each image into a normalized representation, measure how close it is to known normal examples, and then learn a separate monotone mapping from this distance-based score to an anomaly probability.

The implementation adds several layers that make the project more scientifically interesting than a minimal kNN detector. Dataset reconstruction is paired with SHA-256 verification, the reference and calibration pools are separated, synthetic anomalies are introduced solely for calibration, two distinct calibration models are compared, calibration quality is evaluated with ECE and Brier score, risk-coverage behavior is measured, reference size and neighborhood size are varied, and controlled image corruptions are used to probe distribution-shift behavior.

The most important interpretive point is that these mechanisms improve the structure of the experiment, not automatically the strength of its scientific conclusions. Calibration remains dependent on the synthetic positive distribution, robustness remains conditional on the selected corruption family, and reproducibility remains partial because the environment and model artifacts are not fully pinned. The source is therefore best understood as a well-structured research prototype whose strongest contribution is the integration of anomaly scoring, calibration, and stress testing into one leakage-aware computational workflow.

For an academic software portfolio, the implementation demonstrates several valuable capabilities: careful control of data boundaries, practical use of modern pretrained vision models, numerical

feature processing, non-parametric anomaly scoring, probabilistic calibration, evaluation design, robustness analysis, artifact management, and attention to reproducibility. The report shows how those components fit together and, equally importantly, where the code's methodological assumptions and implementation constraints should remain visible to a critical technical reader.